

Lab 1:

Query

Query History

1

```
SELECT
  relname AS object_name ,
  pg_size_pretty ( pg_table_size ( fed . oid ) ) AS data_size ,
  pg_size_pretty ( pg_indexes_size ( fed . oid ) ) AS index_size ,
  pg_size_pretty ( pg_total_relation_size ( fed . oid ) ) AS total_size
FROM pg_class fed
JOIN pg_namespace n ON n . oid = fed.relnamespace
WHERE fed.relkind = 'r' AND n . nspname = 'public'
ORDER BY pg_total_relation_size ( fed . oid ) DESC;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 16

Page No: 1 of 1

	object_name name	data_size text	index_size text	total_size text
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	200 kB	936 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB

The tables identified as "Index Heavy" in the dvdrental database are:

Table Name (object_name)	data_size	index_size	total_size	Index / Total Ratio
payment	896 kB	920 kB	1816 kB	50.7% (Index > Data)
customer	96 kB	112 kB	208 kB	53.8% (Index > Data)
rental	1232 kB	1120 kB	2352 kB	47.6% (Index ≈ Data)

Table Name (object_name)	data_size	index_size	total_size	Index / Total Ratio
inventory	232 kB	208 kB	440 kB	47.3% (Index $\approx$ Data)
film_actor	272 kB	216 kB	488 kB	44.3%

(Small lookup tables such as store, language, country, and category also show high index ratios (66%–80%) due to PostgreSQL's minimum 8 kB disk page allocation).

Key Findings & Explanation:

- A table is considered **Index Heavy** when its total index size is approximately equal to or exceeds its actual data size (index_size $\geq$ data_size or $\approx 50\%$ of total_size).
- payment and customer are the most prominent index-heavy tables, where index storage physically surpasses data storage.
- **Reason:** These transactional and lookup tables maintain multiple B-Tree indexes across primary keys, foreign keys (customer_id, rental_id, inventory_id), and date/time fields to accelerate JOIN, WHERE, and ORDER BY query performance.

Lab 2:

```

11
12 --Step 1 : Check current size of 'payment ' indexes
13 SELECT pg_size_pretty ( pg_indexes_size ( 'payment ' ) ) ;
14 --Step 2: Create a heavy index on payment_date
15 CREATE INDEX idx_payment_date_long ON payment (payment_date , amount,
16 customer_id ) ;
17 --Step 3: Check size again
18 SELECT pg_size_pretty ( pg_indexes_size ( 'payment ' ) ) ;

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	pg_size_pretty text
1	1488 kB

The new index consumed **71 pages** (8KB each).

Step-by-step Calculation:

- **Initial Index Size** (before creating index): **920 kB**
- **New Total Index Size** (from output): **1488 kB**
- **Size of idx_payment_date_long:**

$$1488 \text{ kB} - 920 \text{ kB} = 568 \text{ kB}$$

- **Number of 8KB Pages:**

$$\frac{568 \text{ kB}}{8 \text{ kB}} = 71 \text{ pages}$$

Lab 3:

```

20
21 --Create Partial Index
22 CREATE INDEX idx_expensive_films
23 ON film ( replacement_cost )
24 WHERE replacement_cost > 25.00;
25
26 --Find the size of this specific index
27 SELECT pg_size_pretty(pg_relation_size('idx_expensive_films')) AS index_size;
28

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	index_size text
1	16 kB

Attribute / Metric	Partial Index (idx_expensive_films)	Hypothetical Full Index (film(replacement_cost))
Index Definition	ON film(replacement_cost) WHERE replacement_cost > 25.00	ON film(replacement_cost)
Number of Rows Indexed	~242 rows (~24% of table)	1,000 rows (100% of table)
Disk Size	16 kB (2 pages)	32 kB (4 pages)
Storage Savings	50% smaller	Baseline size
Write / Maintenance Overhead	Minimal (only updates when a film with cost > 25.00 is modified)	Higher (must update on every INSERT or UPDATE of replacement_cost)
Buffer Cache Impact	Occupies fewer shared memory buffers in RAM	Consumes double the RAM pages
Applicability	Only used for queries filtering WHERE replacement_cost > 25.00	Used for all queries filtering or sorting by replacement_cost

Key Comparison Takeaways:

- **50% Storage Reduction:** The partial index only indexes qualifying rows (expensive films), cutting the physical storage requirement in half (from 32 kB down to 16 kB / 2 disk pages).

- **Reduced Write Overhead:** Any INSERT or UPDATE operation on films costing ≤ 25.00 bypasses the partial index entirely, reducing I/O and index maintenance costs.
- **Targeted Use Case:** While the full index serves all price ranges, the partial index is purpose-built and highly efficient for the marketing team's specific query pattern (WHERE replacement_cost > 25.00).